

Lecture 3: Merge Sort

Siyu Chen

siyu.chen.sc3648@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

A recursive sorting procedure can become substantially faster when it reduces a problem of size n to two problems of roughly half the size rather than to one problem of size $n - 1$. Merge sort uses this balanced decomposition and a linear-comparison merge step. We prove that the procedure halts, prove its correctness by strong induction, and derive a worst-case comparison count of $\Theta(n \log n)$. We then show why a general comparison sort cannot improve this worst-case order to linear time.

AI use declaration. We used GPT-5.6 Sol to research as well as polish the writing in the notes.

1 Motivation

Consider a list of n records that must be arranged by a numeric key, such as patient measurements ordered from smallest to largest. A sorting procedure can accomplish this by comparing records and relocating them. The way the procedure decomposes the task determines how much work remains after each step.

One possible decomposition removes only one record from consideration at a time. After one pass through a list of n records, the remaining sorting problem may have size $n - 1$. Repeating that pattern produces subproblems of sizes

$$n, n - 1, n - 2, \dots, 1,$$

and the number of comparisons can grow quadratically with n .

A different decomposition is possible. Instead of reducing a problem of size n to one of size $n - 1$, divide the records into two groups whose sizes differ by at most one. Sort each group recursively, and then combine the two sorted groups into a single sorted list. The recursive subproblems now have sizes approximately $n/2$ rather than $n - 1$.

This chapter develops that procedure, called *merge sort*. We ask the three questions that apply to any recursive procedure: does it halt, does it return the required output, and what does it cost? The final question leads to a second one. Merge sort requires order $n \log n$ comparisons in the worst case. Can a comparison-based sorting procedure do asymptotically better and reach linear time?

2 Setup and Notation

Prerequisites. This chapter assumes summation notation, logarithms, and proof by induction. The forms of induction used in the correctness argument are stated when they are needed. No prior knowledge of merge sort is assumed.

Lists and slices. Let A be a list containing n records, indexed from zero as $A[0], \dots, A[n - 1]$. For integers $0 \leq i \leq j \leq n$, write $A[i:j]$ for the slice $A[i], A[i + 1], \dots, A[j - 1]$. The right endpoint is excluded, so $A[i:j]$ has $j - i$ records. In particular, $A[0:n]$ denotes the entire list.

The records are drawn from a *totally ordered set*: for any two records x and y , exactly one of $x < y$, $x = y$, or $x > y$ holds, and the order is transitive.

Definition 1 (The sorting problem). The input is a list A of n records drawn from a totally ordered set. The output is a permutation of those same records such that

$$A[0] \leq A[1] \leq \dots \leq A[n - 1].$$

The permutation requirement means that sorting preserves the records and their multiplicities. Producing an ordered list is not sufficient if a record has been removed, duplicated, or replaced.

Subproblem size. For a call acting on $A[\ell:r]$, define

$$n := r - \ell.$$

When $n \geq 2$, let

$$m := \ell + \left\lfloor \frac{n}{2} \right\rfloor.$$

Then $A[\ell:m]$ and $A[m:r]$ have sizes

$$\left\lfloor \frac{n}{2} \right\rfloor \quad \text{and} \quad \left\lceil \frac{n}{2} \right\rceil,$$

respectively, so the two sizes differ by at most one.

Cost. We count comparisons between records rather than seconds. Seconds depend on the machine and implementation, while a comparison count isolates the work required by the method. Space is memory used beyond the input and the output being constructed.

Definition 2 (Asymptotic notation). Let f and g map the positive integers to the non-negative reals. The set $O(g(n))$ contains functions f for which there exist positive constants c and n_0 with $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$. The set $\Omega(g(n))$ contains functions f for which there exist positive constants c and n_0 with $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0$. Their intersection is $\Theta(g(n))$.

Definition 3 (Comparison sort). A comparison sort obtains information about the relative order of records only through pairwise comparisons of their keys.

This distinction will matter in Section 8. The lower bound proved there applies to comparison sorting and does not rule out methods that exploit additional structure in the keys.

3 Divide, Conquer, and Merge

Consider first a list of five records. A balanced split cannot divide five into two equal integers, so it produces subproblems of sizes two and three:

$$5 = \left\lfloor \frac{5}{2} \right\rfloor + \left\lceil \frac{5}{2} \right\rceil = 2 + 3.$$

The size-three problem is then split into sizes one and two, and every size-two problem is split into two size-one problems. Figure 1 shows the resulting decomposition.

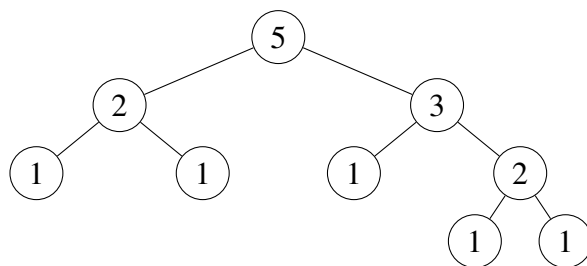


Figure 1: Balanced decomposition of a size-five sorting problem.

A one-record list is already sorted. The remaining task is therefore to combine two sorted sublists into one sorted list.

Merging two sorted lists. Let A_1 and A_2 be sorted lists, and let i and j identify the first unmerged record in each list. If one list has been exhausted, the unmerged suffix of the other can be returned directly. Otherwise the smaller of $A_1[i]$ and $A_2[j]$ is the smallest record remaining in either list and is emitted next.

Algorithm 1 Recursive merge of two sorted lists. The symbol $\parallel$ means concatenation of sequences; the cost analysis counts record comparisons.

```

1: function MERGE( $A_1, A_2, i, j$ )
2:   if  $i = |A_1|$  then
3:     return  $A_2[j:|A_2|]$ 
4:   else if  $j = |A_2|$  then
5:     return  $A_1[i:|A_1|]$ 
6:   else if  $A_1[i] \leq A_2[j]$  then
7:      $R \leftarrow \text{MERGE}(A_1, A_2, i + 1, j)$ 
8:     return  $\langle A_1[i] \rangle \parallel R$ 
9:   else
10:     $R \leftarrow \text{MERGE}(A_1, A_2, i, j + 1)$ 
11:    return  $\langle A_2[j] \rangle \parallel R$ 
12:   end if
13: end function

```

Remark 1 (What the pseudocode costs). Algorithm 1 is written to mirror the recursive construction used in the lecture, and the analysis below counts comparisons between records. A literal implementation using repeated array or Python-list concatenation would incur additional copying costs. A standard implementation instead writes each selected record once into an output array, preserving the same comparison logic while achieving linear merge time.

Merge sort. The sorting procedure applies the same idea recursively to the two halves and then merges their sorted outputs.

Algorithm 2 Merge sort on the half-open slice $A[\ell:r]$.

```

1: function MERGESORT( $A, \ell, r$ )
2:    $n \leftarrow r - \ell$ 
3:   if  $n \leq 1$  then
4:     return  $A[\ell:r]$ 
5:   end if
6:    $m \leftarrow \ell + \lfloor n/2 \rfloor$ 
7:    $A_1 \leftarrow \text{MERGESORT}(A, \ell, m)$ 
8:    $A_2 \leftarrow \text{MERGESORT}(A, m, r)$ 
9:   return MERGE( $A_1, A_2, 0, 0$ )
10: end function

```

The decomposition is a divide-and-conquer procedure: divide the original problem into smaller problems, solve those problems recursively, and combine their solutions. Merge sort is a standard example of this paradigm (Cormen et al., 2022).

4 Halting

A recursive definition does not by itself guarantee that a chain of calls will end. We use a decreasing integer-valued measure to establish that both procedures halt.

Definition 4 (Measure). A measure for a recursive procedure is a non-negative integer computed from the arguments at every call and strictly decreased by every recursive call.

Theorem 1 (Halting). *For finite input lists, both MERGE and MERGESORT halt.*

Proof. For $\text{MERGE}(A_1, A_2, i, j)$, take

$$d := (|A_1| - i) + (|A_2| - j),$$

the number of unmerged records. If neither base case applies, exactly one of i and j increases by one. The next recursive call therefore has measure $d - 1$. Since d is a non-negative integer, the recursion reaches a call in which one list is exhausted.

For $\text{MERGESORT}(A, \ell, r)$, take $n := r - \ell$. The procedure returns immediately when $n \leq 1$. If $n \geq 2$, the recursive calls have sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, both strictly smaller than n . Thus every branch reaches a base case. The final call to MERGE halts by the preceding paragraph. $\square$

5 Correctness

We first establish what the merge operation guarantees. The sorting theorem then uses that result as its combining step.

Definition 5 (Strong induction). Let $P(n)$ be a claim about every non-negative integer n . If the assumption that $P(k)$ holds for every $k < n$ implies $P(n)$, together with valid base cases, then $P(n)$ holds for every n .

Lemma 2 (Correctness of merge). *Suppose A_1 and A_2 are each in non-decreasing order. Then $\text{MERGE}(A_1, A_2, i, j)$ returns a non-decreasing list containing exactly the records in $A_1[i:|A_1|]$ and $A_2[j:|A_2|]$, counted with multiplicity.*

Proof. Use strong induction on

$$d := (|A_1| - i) + (|A_2| - j),$$

the number of records that remain to be merged.

Base case. If $i = |A_1|$, the procedure returns the suffix $A_2[j:|A_2|]$, which is already in non-decreasing order and contains exactly the remaining records. The case $j = |A_2|$ is symmetric.

Inductive hypothesis. Assume the claim holds whenever fewer than d records remain.

Inductive step. Suppose both lists contain an unmerged record. If $A_1[i] \leq A_2[j]$, then $A_1[i]$ is no larger than any later record in A_1 , because A_1 is sorted, and it is no larger than any remaining record in A_2 , because $A_1[i] \leq A_2[j]$ and A_2 is sorted. Thus $A_1[i]$ is the smallest record remaining in either list. The recursive call $\text{MERGE}(A_1, A_2, i + 1, j)$ has $d - 1$ remaining records, so by the inductive hypothesis it returns those records in non-decreasing order and with the correct multiplicities. Prepending $A_1[i]$ preserves both properties. The case $A_1[i] > A_2[j]$ is symmetric, with $A_2[j]$ emitted first. $\square$

Theorem 3 (Correctness of merge sort). *For every finite list A , $\text{MERGESORT}(A, 0, |A|)$ returns a permutation of A in non-decreasing order.*

Proof. Use strong induction on $n := |A|$.

Base case ($n \leq 1$). A list containing at most one record is already in non-decreasing order, and the procedure returns that record set unchanged.

Inductive hypothesis. Assume the theorem holds for every input size $k < n$.

Inductive step ($n \geq 2$). The procedure splits the list into two slices of sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Both sizes are strictly smaller than n , so the inductive hypothesis says that the two recursive calls return sorted permutations of their respective input slices. Lemma 2 then says that merging those outputs returns all records from both slices, with their multiplicities preserved, in non-decreasing order. The two slices partition the original input, so the merged result is a sorted permutation of the original list. $\square$

The use of strong induction matches the shape of the procedure: merge sort recurses on subproblems of sizes approximately $n/2$, not only on the immediate predecessor $n - 1$.

6 Cost

Cost of merge. Suppose the two input lists contain p and q records, for a total of $n = p + q$. Each comparison emits one record and increases either i or j by one. Once one list is exhausted, the rest of the other list is returned without further record comparisons. In the worst case neither list is exhausted until $n - 1$ records have been emitted, so the worst-case number of record comparisons is $n - 1 \in \Theta(n)$.

Cost of merge sort. For arbitrary n , the worst-case comparison count satisfies

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + \Theta(n).$$

To expose the growth rate, first suppose $n = 2^d$. Then

$$T(n) = 2T(n/2) + \Theta(n).$$

At the root there is one subproblem of size n , so the merge work is $\Theta(n)$. At the next level there are two subproblems of size $n/2$, for a total merge work of

$$2 \Theta(n/2) = \Theta(n).$$

At level k there are 2^k subproblems of size $n/2^k$, and the total merge work remains

$$2^k \Theta\left(\frac{n}{2^k}\right) = \Theta(n).$$

The recursion reaches size one when

$$\frac{n}{2^d} = 1,$$

so $d = \log_2 n$. There are therefore $\Theta(\log n)$ levels, each with $\Theta(n)$ total merge work, giving

$$T(n) \in \Theta(n \log n).$$

The floor and ceiling terms for non-powers of two alter subproblem sizes by at most one and do not change this asymptotic order. The same recurrence analysis is standard for merge sort ([Cormen et al., 2022](#)).

Space. The recursive merge in Algorithm 1 can make $\Theta(n)$ nested calls while an output sequence of length n is being formed. A standard array-based implementation uses $\Theta(n)$ auxiliary storage for the merged output, while the merge-sort recursion itself has depth $\Theta(\log n)$ ([Cormen et al., 2022](#)). Thus the usual merge-sort implementation uses $\Theta(n)$ auxiliary space. The exact allocation behavior of a literal language-specific translation depends on how sequence construction is implemented.

7 Worked Example

For illustration, take the five values

$$A = \langle 7, 2, 6, 3, 5 \rangle.$$

The first split has sizes two and three:

$$\langle 7, 2 \rangle \quad \text{and} \quad \langle 6, 3, 5 \rangle.$$

The left side splits into $\langle 7 \rangle$ and $\langle 2 \rangle$, which merge to $\langle 2, 7 \rangle$. The right side splits into $\langle 6 \rangle$ and $\langle 3, 5 \rangle$; the second of these becomes $\langle 3 \rangle$ and $\langle 5 \rangle$ and merges to $\langle 3, 5 \rangle$. Merging $\langle 6 \rangle$ with $\langle 3, 5 \rangle$ gives $\langle 3, 5, 6 \rangle$. The final merge is

$$\langle 2, 7 \rangle \quad \text{with} \quad \langle 3, 5, 6 \rangle,$$

which produces

$$\langle 2, 3, 5, 6, 7 \rangle.$$

subproblem	split or merge	result
$\langle 7, 2 \rangle$	$\langle 7 \rangle, \langle 2 \rangle$	$\langle 2, 7 \rangle$
$\langle 3, 5 \rangle$	$\langle 3 \rangle, \langle 5 \rangle$	$\langle 3, 5 \rangle$
$\langle 6, 3, 5 \rangle$	$\langle 6 \rangle, \langle 3, 5 \rangle$	$\langle 3, 5, 6 \rangle$
whole list	$\langle 2, 7 \rangle, \langle 3, 5, 6 \rangle$	$\langle 2, 3, 5, 6, 7 \rangle$

The example also shows why an odd input size causes no difficulty: the initial five-record problem is divided into subproblems of sizes two and three, and both are strictly smaller than five.

8 Beyond the Lecture: Can Comparison Sorting Be Linear?

The cost analysis leaves one final question: after improving quadratic comparison growth to $\Theta(n \log n)$, could a different comparison-based method reduce the worst-case order again, all the way to $\Theta(n)$? For general worst-case comparison sorting, the answer is no. The standard decision-tree argument below gives the lower bound (Cormen et al., 2022).

Theorem 4 (Comparison-sorting lower bound). *Every deterministic comparison sort requires $\Omega(n \log n)$ comparisons in the worst case when sorting n distinct records.*

Proof. There are $n!$ possible input orderings of n distinct records. A correct sorting procedure must distinguish among all of them. Because the records are distinct, each pairwise ordering comparison has two relevant outcomes. A deterministic comparison sort can therefore be represented as a binary decision tree whose leaves correspond to possible orderings. A binary tree of height h has at most 2^h leaves, so

$$2^h \geq n!.$$

Taking base-two logarithms gives

$$h \geq \log_2(n!).$$

For $n \geq 2$, at least $\lfloor n/2 \rfloor$ factors in $n!$ are at least $n/2$, so

$$n! \geq \left(\frac{n}{2}\right)^{\lfloor n/2 \rfloor}.$$

Therefore

$$\log_2(n!) \geq \left\lfloor \frac{n}{2} \right\rfloor \log_2\left(\frac{n}{2}\right) \in \Omega(n \log n).$$

Thus the height of the decision tree, and hence the worst-case number of comparisons, is $\Omega(n \log n)$. $\square$

Merge sort has a worst-case upper bound of $O(n \log n)$ and every comparison sort has a worst-case lower bound of $\Omega(n \log n)$. Merge sort is therefore asymptotically optimal among comparison sorts.

Remark 2 (Why the word “comparison” matters). The theorem does not say that every possible sorting method requires $\Omega(n \log n)$ time. Algorithms that exploit additional structure in the keys can escape the comparison model. For example, counting-style methods can run in linear time when keys lie in a suitably bounded integer range (Cormen et al., 2022). The lower bound is a statement about the information obtained through pairwise comparisons, not an absolute prohibition on linear-time sorting.

9 Summary

Merge sort changes the decomposition of the sorting problem. Instead of reducing size n to size $n - 1$, it reduces the task to two subproblems of sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. The merge operation is correct because the smaller of the two first unmerged records is the smallest record remaining overall, and it halts because one unmerged record is removed at every recursive call.

The sorting procedure halts because every recursive subproblem is strictly smaller than its parent, and it is correct by strong induction together with the merge lemma. Each recursion level performs $\Theta(n)$ total merge work, and there are $\Theta(\log n)$ levels, so the worst-case comparison count is $\Theta(n \log n)$. A binary decision-tree argument gives a matching $\Omega(n \log n)$ lower bound for general comparison sorting, so merge sort is asymptotically optimal within that model.

Further Reading

Cormen et al. (2022) give a standard treatment of merge sort, divide-and-conquer recurrences, and the decision-tree lower bound for comparison sorting. Knuth (1998) provide a broader reference on sorting and searching, including detailed analysis of comparison-based sorting methods.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.